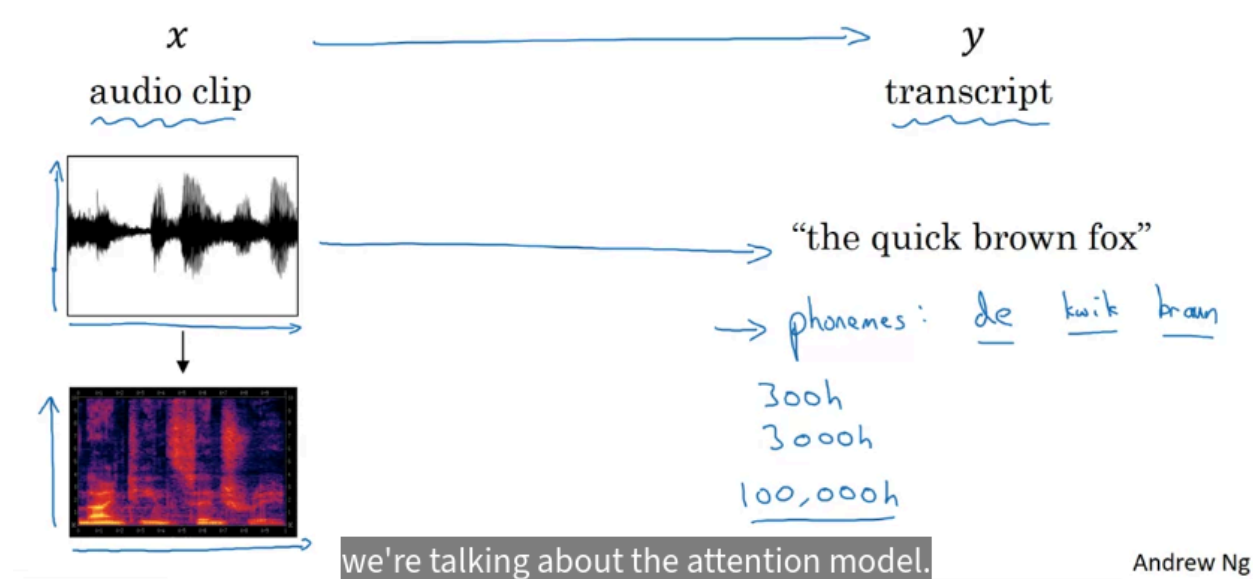


Speech Recognition - Audio Data

Speech Recognition

Speech recognition problem



Speech Recognition with Sequence-to-Sequence Models

Speech recognition is the task of converting an audio signal into text.

Given:

x = audio clip

the goal is to predict:

y = text transcript

For example:

Audio:

| "the quick brown fox"

Output:

| the quick brown fox

This can be formulated as a sequence-to-sequence problem because both the audio and transcript are sequences.

Audio Representation

A microphone records very small changes in air pressure over time.

A raw waveform can be represented as:

Amplitude vs. Time

where:

- Horizontal axis = time
- Vertical axis = air pressure / signal amplitude

However, speech recognition systems often do not directly use the raw waveform.

A common preprocessing step is to convert the signal into a frequency-based representation.

Spectrogram

A **spectrogram** represents how the frequency content of an audio signal changes over time.

Its axes are:

- Horizontal axis = time

- Vertical axis = frequency
- Color or intensity = energy at each frequency

Conceptually:

Raw waveform → Spectrogram → Speech recognition model

The spectrogram makes important acoustic structures easier to represent.

Filterbank Features

Another common audio representation is **filterbank features**.

These features summarize the amount of energy in different frequency bands at each time step.

Conceptually:

Audio waveform

→ Frequency analysis

→ Filterbank features

→ Neural network

This preprocessing is somewhat similar to how the human ear separates sounds according to frequency.

Traditional Speech Recognition

Older speech recognition systems often relied heavily on **phonemes**.

A phoneme is a basic unit of sound in a language.

For example, a sentence such as:

the quick brown fox

could first be represented as a sequence of phonemes.

The pipeline was roughly:

Audio

→ Acoustic features

→ Phonemes

→ Words

→ Sentence

Phonemes acted as a manually designed intermediate representation.

End-to-End Speech Recognition

Modern deep learning allows the intermediate phoneme representation to be removed.

Instead:

Audio → Neural network → Transcript

This is called **end-to-end speech recognition**.

The model learns the mapping directly from audio to text.

Therefore, it does not necessarily require manually engineered representations such as phonemes.

Importance of Large Datasets

A major factor in the success of modern speech recognition is the availability of much larger datasets.

A speech dataset contains pairs:

(x, y)

where:

- x = audio recording

- y = corresponding transcript

The lecture gives approximate scales such as:

- Hundreds of hours for smaller academic datasets
- Several thousand hours for larger research datasets
- More than 10,000 hours for commercial systems
- Sometimes over 100,000 hours

Large transcribed datasets combined with deep learning have significantly improved speech recognition performance.

Two Approaches to Speech Recognition

The lecture introduces two major approaches:

1. Attention-based sequence-to-sequence models
2. Connectionist Temporal Classification

Both transform an audio sequence into a shorter text sequence.

Attention-Based Speech Recognition

An attention-based model can use:

Audio features → Encoder → Attention → Decoder → Transcript

The input consists of many audio time frames.

The decoder generates text one token at a time.

For example:

Audio frames:

$x^1, x^2, \dots, x^{T_x}$

Output:

| the → quick → brown → fox → EOS

Attention helps the decoder decide which parts of the audio are relevant when generating each output token.

Conceptually:

Current output token

→ Attend to relevant audio frames

→ Predict next text token

Why Attention Is Useful

Speech sequences can be long.

A fixed-size context vector would need to compress the entire audio signal.

Attention instead allows the decoder to dynamically access different parts of the encoded audio sequence.

For example:

When generating:

| quick

the model may focus mainly on the region of audio corresponding to the spoken word “quick.”

Input and Output Length Mismatch

Speech recognition has a major sequence-length mismatch.

Usually:

$T_x \gg T_y$

where:

- T_x = number of audio time steps
- T_y = number of output characters or words

For example, suppose:

Audio duration = 10 seconds

Feature rate = 100 Hz

Then:

$$T_x = 10 \times 100 = 1000$$

So the model receives approximately:

1000 audio frames

However, the transcript:

| the quick brown fox

contains only about 19 characters including spaces.

Therefore:

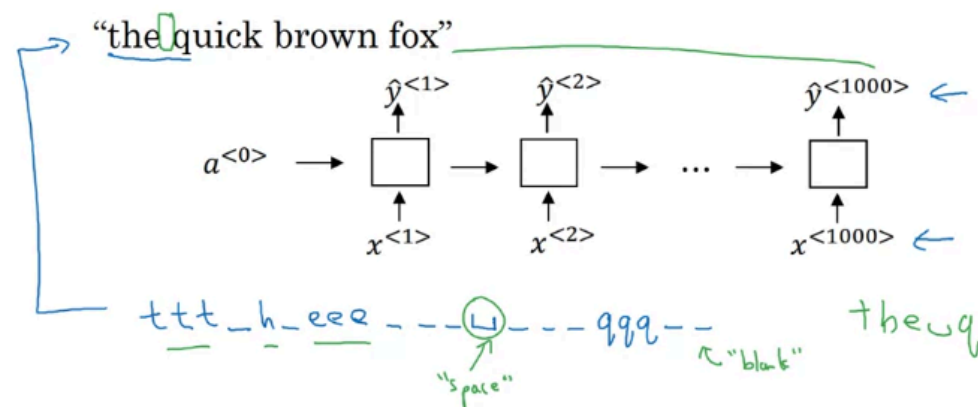
1000 input steps $\gg$ 19 output characters

This mismatch must be handled by the speech recognition architecture.

Connectionist Temporal Classification

CTC cost for speech recognition

(Connectionist temporal classification)



Basic rule: collapse repeated characters not separated by "blank" to have a thousand outputs by repeating characters allow the times. Andrew Ng

Another approach is **CTC**, which stands for:

Connectionist Temporal Classification

CTC allows a neural network to produce a long output sequence while still representing a much shorter transcript.

It introduces a special symbol:

blank

We can represent it as:

–

Important:

Blank  is different from the actual space character .

Basic CTC Idea

Suppose the target transcript begins with:

| the q...

The network may output something like:

t t t _ h _ e e e _ _ _ q q q _ _

This sequence is much longer than the final transcript.

CTC applies a collapsing rule to convert it into the final text.

CTC Collapse Rule

CTC uses two main operations:

1. Collapse Consecutive Repeated Characters

Repeated characters that are not separated by blanks are collapsed.

Example:

t t t → t

h h h → h

e e e → e

q q q → q

2. Remove Blank Tokens

Blank symbols are removed.

Example:

t _ h _ e

becomes:

the

Therefore:

t t t _ h _ e e e _ _ _

can collapse into:

the

Important Role of the Blank Token

The blank token helps distinguish repeated output characters.

For example:

Suppose the target is:

| hello

There are two consecutive **l** characters.

If the network outputs:

l l

CTC would collapse them into:

l

To represent two separate **l** characters, the network can output:

l _ l

After CTC collapsing:

l _ l → ll

Therefore, blanks allow the model to represent repeated characters correctly.

Why CTC Works for Speech

Suppose there are:

1000 audio frames

but only:

19 characters in the transcript.

The CTC model can output one prediction for every input time step:

$y^1, y^2, \dots, y^{1000}$

Most predictions may be:

- Blank tokens
- Repeated characters

After collapsing them, the sequence becomes the much shorter transcript.

Conceptually:

1000 frame-level outputs

→ Collapse repeated symbols

→ Remove blanks

→ 19-character transcript

This solves the input-output length mismatch.

Example

Target:

| the

Possible CTC output:

t t t _ h h h _ e e e

Collapse repeated characters:

t _ h _ e

Remove blanks:

the

Another valid output could be:

t _ h h _ e _

which also collapses to:

the

Therefore, many different frame-level sequences can correspond to the same final transcript.

CTC Does Not Require Exact Alignment

One important consequence is that the training data does not need to specify exactly:

- Which audio frame corresponds to **t**
- Which frame corresponds to **h**
- Which frame corresponds to **e**

The model only needs:

Audio ↔ Transcript

For example:

Audio clip ↔ "the quick brown fox"

CTC considers many possible alignments between the audio frames and transcript.

This is especially useful because manually aligning every character or phoneme with exact audio timestamps would be expensive.

Neural Network for CTC

The transcript describes a recurrent architecture such as:

Audio features

- Deep recurrent network
- Frame-level character probabilities
- CTC decoding

Possible recurrent units include:

- Bidirectional RNN
- Bidirectional GRU
- Bidirectional LSTM

In practice, deeper bidirectional recurrent architectures have historically been used.

At every time step, the model predicts probabilities over:

- Characters
- Space
- Blank
- Possibly other symbols

CTC Output Vocabulary

For character-level speech recognition, the output vocabulary may include:

- a–z
- Space
- Apostrophe
- Other punctuation
- Blank

The blank token is an artificial symbol used internally by CTC.

It does not appear in the final transcript.

CTC Training Objective

Rather than forcing one exact frame-level alignment, CTC considers all valid alignments that collapse to the correct transcript.

Conceptually:

$$P(y \mid x) = \sum P(\text{alignment} \mid x)$$

where the sum is taken over all alignments that collapse into the target transcript y.

The model is trained to maximize the total probability of all valid alignments.

This is the central idea behind the **CTC loss**.

CTC vs. Attention-Based Models

Aspect	CTC	Attention-based Seq2Seq
Input	Audio sequence	Audio sequence
Output	Transcript	Transcript
Alignment	Implicit, monotonic	Learned through attention
Blank token	Required	Not required
Decoder	Not necessarily autoregressive	Usually autoregressive

Aspect	CTC	Attention-based Seq2Seq
Output length	Handled by CTC collapse	Generated token by token
Exact frame alignment needed	No	No
Main idea	Collapse long output into short transcript	Attend to relevant audio regions

Monotonic Nature of Speech

Speech recognition usually has a mostly monotonic relationship between input and output.

If someone says:

the quick brown fox

the acoustic evidence for:

the

usually appears before:

quick

which appears before:

brown

and so on.

CTC naturally fits this type of monotonic sequence alignment.

It does not need to arbitrarily reorder the output sequence.

Attention vs. CTC Intuition

Attention

Think:

For the next output token, which audio frames should I focus on?

CTC

Think:

Predict something at every audio frame, then collapse the long prediction sequence into text.

These are two different ways of solving the same audio-to-text mapping problem.

Historical Architecture

The lecture mentions systems using recurrent networks such as:

Audio features

- Deep Bidirectional RNN / LSTM / GRU
- Character probabilities
- CTC

Deep Speech is an example of this type of approach.

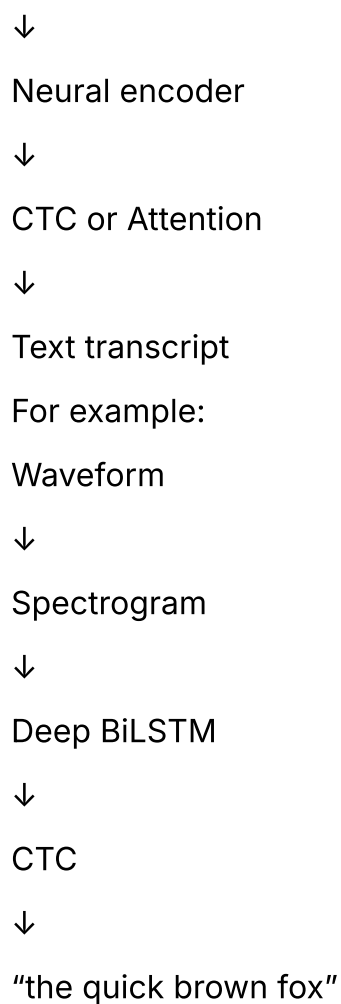
End-to-End ASR Pipeline

A simplified end-to-end speech recognition system can be represented as:

Audio waveform

↓

Spectrogram / Filterbank features



Benefits of End-to-End Speech Recognition

Compared with older phoneme-based pipelines, end-to-end models reduce the need for manually designed intermediate components.

Potential advantages include:

- Simpler overall pipeline
- Direct optimization of audio-to-text mapping
- Less reliance on hand-designed phoneme systems
- Ability to learn acoustic representations automatically
- Improved performance with sufficiently large datasets

However, they generally require substantial amounts of labeled speech data.

Main Challenges

Large Dataset Requirements

Production-scale ASR may require thousands or tens of thousands of hours of transcribed speech.

Long Input Sequences

Speech contains many more acoustic frames than output characters.

For example:

$$T_x = 1000$$

$$T_y \approx 19$$

Alignment

The model must learn which portions of the audio correspond to which output symbols.

Attention and CTC solve this in different ways.

Computational Cost

Long audio sequences require significant computation and memory, especially for deep bidirectional models.

Key Equations

Speech Recognition Objective

Given audio x :

$$\hat{y} = \operatorname{argmax}_y P(y \mid x)$$

The goal is to find the most likely transcript.

Input Length Example

At 100 feature frames per second:

$$10 \text{ seconds} \times 100 \text{ frames/second} = 1000 \text{ frames}$$

Sequence Length Relationship

Usually:

$$T_x \gg T_y$$

CTC Objective

Conceptually:

$$P(y \mid x)$$

$$= \sum_{a \in A(y)} P(a \mid x)$$

where:

- $A(y)$ = all frame-level alignments that collapse to transcript y
- a = one possible alignment

The model therefore does not need one manually specified alignment.

Key Takeaways

- Speech recognition maps an audio sequence to a text transcript.
- Audio can be represented as raw waveform, spectrogram, or filterbank features.
- Spectrograms represent frequency energy over time.
- Older systems relied heavily on manually defined phonemes.
- End-to-end deep learning can map audio directly to text.
- Large transcribed speech datasets have been crucial to modern ASR progress.
- Attention-based Seq2Seq models can be used for speech recognition.
- CTC is another important approach.
- Speech usually has many more audio frames than output characters.
- CTC handles this mismatch using repeated characters and a special blank symbol.
- Repeated symbols are collapsed and blanks are removed.
- Many CTC alignments can represent the same transcript.
- CTC therefore does not require manually aligned frame-level labels.
- CTC and attention provide two different approaches to audio-to-text sequence modeling.

Further Learning

CTC Loss

Study the mathematical details of:

$$P(y \mid x) = \sum_{a \in A(y)} P(a \mid x)$$

Important concepts include:

- Alignment paths
- Blank symbols

- Collapse function
- Forward-backward algorithm
- CTC decoding

Greedy CTC Decoding

A basic decoder can:

1. Choose the highest-probability character at every frame.
2. Collapse consecutive repetitions.
3. Remove blank symbols.

This is called **greedy CTC decoding**.

CTC Beam Search

More advanced decoding can maintain multiple candidate transcripts.

Useful topics:

- Beam search
- Prefix beam search
- Language-model fusion
- Acoustic score
- Language-model score

Modern Audio Features

Useful topics include:

- STFT
- Mel spectrogram
- Log-Mel filterbanks
- MFCC
- Raw waveform encoders

Modern ASR Architectures

Later approaches extend these ideas with architectures such as:

- Transformer encoders
- Conformer
- RNN-Transducer
- Attention-based encoder-decoder
- Self-supervised speech encoders

Self-Supervised Speech Representation Learning

A useful continuation is to study models that learn speech representations from large amounts of unlabeled audio before fine-tuning for ASR.

Important concepts include:

- Speech representation learning
- Pretraining
- Fine-tuning
- Contrastive objectives
- Masked prediction

Trigger-Word Detection

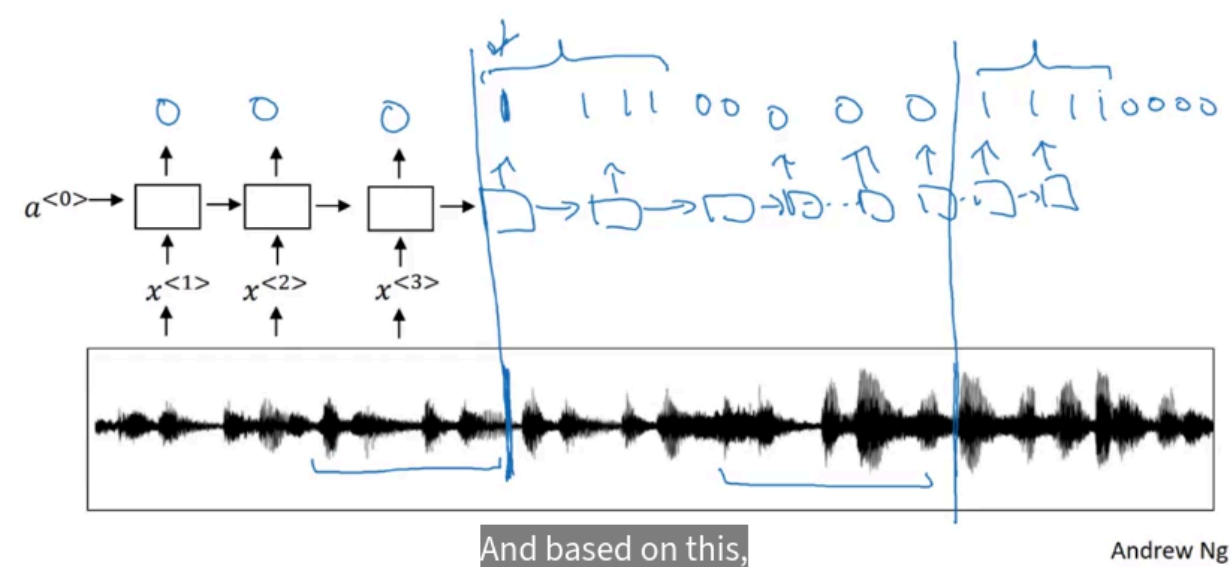
The next lesson focuses on a simpler speech task:

▮ Detect whether a specific keyword or trigger phrase occurs in an audio stream.

Unlike full ASR, trigger-word detection can often be trained using significantly less data.

Trigger Word Detection

Trigger word detection algorithm



Trigger Word Detection

A **trigger word detection system** listens continuously for a specific word or phrase and activates a device or application when that phrase is detected.

Common examples include:

- "Alexa"
- "Hey Siri"
- "Okay Google"
- "Xiaodu ni hao"

The general goal is:

Audio stream → Detect trigger phrase → Activate system

For example:

▮ "Alexa, what time is it?"

The device should remain inactive until it detects:

▮ "Alexa"

Basic Idea

Trigger word detection can be formulated as a sequence-labeling problem.

The pipeline is:

Audio clip

→ Audio features

→ RNN

→ Binary outputs over time

The model processes an audio sequence and predicts at every time step whether the trigger word has just been detected.

Audio Preprocessing

As in speech recognition, raw audio can first be transformed into acoustic features.

A common pipeline is:

Waveform

→ Spectrogram

→ Feature sequence $x^1, x^2, \dots, x^T$

These features are then passed into a recurrent neural network.

The recurrent model can be:

- Basic RNN
- GRU
- LSTM

Input Sequence

Suppose the audio produces features:

$x^1, x^2, x^3, \dots, x^T$

Each x^t represents the audio information at a specific time step.

The RNN processes these features sequentially.

At every time step it produces an output:

$\hat{y}^t$

which estimates whether the trigger word has just occurred.

Defining the Target Labels

The most important design choice is the target sequence:

$y^1, y^2, \dots, y^T$

This is a binary sequence.

Normally:

$y^t = 0$

when no trigger word has just finished.

When the trigger word finishes at time step t :

$y^t = 1$

So the labels may look like:

0 0 0 0 0 0 1 0 0 0 0 ...

The value 1 appears immediately after the trigger phrase is completed.

Example

Suppose the audio contains:

└ "hello Alexa what time is it"

Before "Alexa" finishes:

$y^t = 0$

Immediately after "Alexa" finishes:

$y^t = 1$

Then the labels return to:

$y^t = 0$

If the trigger word appears again later:

| "Alexa ... Alexa ..."

the target sequence contains another positive label after the second occurrence.

Conceptually:

Audio:

_____ Alexa _____ Alexa _____

Labels:

00000010000000000000100000

Why Label After the Trigger Word?

The model should activate only after enough audio has been observed to determine that the complete trigger phrase occurred.

For example, the system cannot confidently recognize:

| "Alexa"

after hearing only:

| "Ale..."

So the positive target is placed around the point where the trigger word has finished.

RNN Formulation

The recurrent network receives acoustic features:

$x^1 \rightarrow x^2 \rightarrow x^3 \rightarrow \dots \rightarrow x^T$

and produces:

$\hat{y}^1 \rightarrow \hat{y}^2 \rightarrow \hat{y}^3 \rightarrow \dots \rightarrow \hat{y}^T$

Each output may be interpreted as:

$\hat{y}^t \approx P(\text{trigger word just finished at time } t)$

The system activates when this probability exceeds some threshold.

For example:

If:

$\hat{y}^t > 0.5$

then:

Trigger detected

In practice, the threshold can be tuned depending on the desired false-positive and false-negative tradeoff.

Class Imbalance Problem

A major issue with this labeling strategy is that most target values are zero.

For a long audio clip:

0 0 0 0 0 0 0 0 1 0 0 0 0 ...

there may be hundreds or thousands of zeros for only one positive label.

Therefore:

Number of negative examples $\gg$ Number of positive examples

This creates a highly imbalanced training dataset.

Extending the Positive Region

A simple practical trick is to label several consecutive time steps as 1 after the trigger word.

Instead of:

000000010000000

use something like:

000000011111000

So after the trigger word finishes:

$y^t = 1$

for several time steps.

Then it returns to zero.

Why Multiple Positive Labels Help

Using several positive labels increases the number of positive training examples.

This can:

- Reduce class imbalance
- Make optimization easier
- Give the model more opportunity to detect the trigger
- Reduce sensitivity to an exact single-frame boundary

However, the lecture emphasizes that this is somewhat of a **hack** rather than a theoretically necessary part of the model.

Example with Multiple Trigger Words

Suppose an audio sequence contains the trigger word twice.

The target may look like:

Audio:

——— Trigger ————— Trigger ———

Targets:

000000111100000000000001111000

Each group of 1s corresponds to a short period immediately after the trigger phrase has finished.

Training Objective

Trigger word detection is essentially binary classification at every time step.

For each t :

$y^t \in \{0, 1\}$

The model predicts:

$\hat{y}^t \in [0, 1]$

A typical loss is binary cross-entropy:

$L^t = -[y^t \log(\hat{y}^t) + (1 - y^t) \log(1 - \hat{y}^t)]$

The total loss over the sequence can be written as:

$L = \sum_{t=1}^T L^t$

The model is trained to assign high probability to the positive regions following the trigger word.

Inference

During deployment:

1. Continuously record audio.
2. Convert audio into features.
3. Feed features through the recurrent network.
4. Compute $\hat{y}^t$ at every time step.

5. If $\hat{y}^t$ becomes sufficiently large, activate the system.

Conceptually:

Microphone

→ Spectrogram

→ RNN

→ Trigger probability

→ Activation

Streaming Nature of the Task

Unlike a bidirectional task that may require the complete sequence, trigger word detection naturally needs to operate in real time.

The system should detect:

| "Alexa"

without waiting for the user to finish the entire sentence:

| "Alexa, what time is it?"

Therefore, a causal or low-latency model is important for practical systems.

Trigger Word Detection vs. Full Speech Recognition

Trigger word detection is much simpler than complete automatic speech recognition.

Full Speech Recognition

Goal:

Audio → Entire transcript

For example:

| "Alexa, what time is it?"

→

| Alexa what time is it

This requires recognizing many possible words and sentences.

Trigger Word Detection

Goal:

Audio → Is the trigger phrase present?

The model only needs to detect a small predefined vocabulary.

This makes trigger detection possible with significantly less training data than a full production-scale speech recognition system.

Trigger Word Detection vs. Keyword Spotting

Trigger word detection is closely related to **keyword spotting**.

Both attempt to detect specific words or phrases in audio.

A trigger word system usually uses the detected keyword to activate another system.

For example:

Trigger detection:

| "Hey Siri"

then activates:

Speech recognition → Intent understanding → Response

Complete Voice Assistant Pipeline

A voice assistant can conceptually use:

Continuous audio

↓

Trigger word detector

↓

Trigger detected

↓

Speech recognition

↓

Text transcript

↓

Language understanding

↓

Action / response

The trigger detector therefore acts as the first stage.

Its goal is to avoid running the full speech-recognition pipeline unnecessarily.

Why RNNs Are Suitable

Audio is sequential.

The acoustic evidence for a trigger phrase is distributed across many consecutive time steps.

An RNN can maintain information about earlier sounds while processing new audio.

For example:

A → lex → a

Each part contributes evidence toward the complete word:

| Alexa

The recurrent hidden state allows the system to accumulate this evidence over time.

Practical Considerations

False Positives

A false positive occurs when the system activates even though the trigger word was not spoken.

Example:

Someone says a similar-sounding word and the device incorrectly activates.

For consumer devices, too many false positives can be very annoying.

False Negatives

A false negative occurs when the trigger word was spoken but the system fails to detect it.

For example:

| "Hey Siri"

but the device does not respond.

A practical system must balance:

- False positive rate

- False negative rate

Threshold Selection

Suppose the model predicts:

$$\hat{y}^t = P(\text{trigger} \mid \text{audio up to } t)$$

The system may activate when:

$$\hat{y}^t > \text{threshold}$$

A higher threshold usually:

- Reduces false positives
- May increase false negatives

A lower threshold usually:

- Improves sensitivity
- May produce more false positives

Therefore, threshold selection is important during deployment.

Simplified Architecture

A basic trigger word detector can be summarized as:

Audio

↓

Spectrogram features

↓

RNN / GRU / LSTM

↓

$\hat{y}^1, \hat{y}^2, \dots, \hat{y}^T$

↓

Threshold

↓

Trigger / No Trigger

Simplified Pseudocode

```
for audio_frame in audio_stream:
    features = extract_features(audio_frame)

    probability, hidden_state = model(
        features,
        hidden_state
    )

    if probability > threshold:
        activate_device()
```

In practice, the model may operate on chunks of audio rather than individual raw samples.

Training Label Example

Suppose:

$T = 15$

and the trigger phrase finishes at time step 7.

Single-frame labeling:

```
0 0 0 0 0 0 1 0 0 0 0 0 0 0
```

Extended labeling:

```
0 0 0 0 0 0 1 1 1 1 0 0 0 0
```

The second approach reduces the imbalance between positive and negative labels.

Key Equations

Binary Target

$$y^t \in \{0, 1\}$$

Trigger Probability

$$\hat{y}^t = P(\text{trigger completed at } t \mid x^1, \dots, x^t)$$

Binary Cross-Entropy

$$L^t = -[y^t \log(\hat{y}^t) + (1 - y^t) \log(1 - \hat{y}^t)]$$

Sequence Loss

$$L = \sum_{t=1}^T L^t$$

Detection Rule

Trigger if:

$$\hat{y}^t > \text{threshold}$$

Key Takeaways

- Trigger word detection activates a system after hearing a predefined phrase.
- Examples include "Alexa," "Hey Siri," and "Okay Google."
- The task can be modeled as sequence labeling over audio frames.
- Audio is commonly converted into spectrogram-like features.
- These features are passed through an RNN, GRU, or LSTM.
- The output at each time step represents whether the trigger word has just finished.
- Most target labels are zero, creating a class-imbalance problem.
- One practical trick is to assign several consecutive positive labels after each trigger occurrence.
- Trigger word detection requires much less data than full speech recognition.
- In practical systems, false positives, false negatives, and latency are important considerations.
- Trigger word detection can serve as the first stage of a larger voice-assistant pipeline.

Course Recap

This lesson also concludes the major sequence-model topics covered in the course.

Recurrent Models

- Basic RNN
- GRU
- LSTM
- Bidirectional RNN

- Deep RNN

Word Representations

- Word embeddings
- Neural embedding learning
- Word2Vec
- Context-target prediction

Sequence-to-Sequence Modeling

- Encoder-decoder models
- Beam search
- Attention

Audio Modeling

- Speech recognition
- CTC
- Attention-based ASR
- Trigger word detection

Together, these ideas provide the foundations for processing:

- Text
- Speech
- Audio
- Sequential data

Further Learning

Keyword Spotting

Study modern keyword-spotting systems and how they are optimized for:

- Low latency
- Small memory footprint
- Low power consumption
- On-device inference

Class Imbalance

Useful techniques beyond extending positive labels include:

- Positive-class weighting
- Oversampling
- Focal loss
- Hard-negative mining

Evaluation Metrics

For trigger detection, accuracy alone is often insufficient because negatives dominate.

Useful metrics include:

- Precision
- Recall
- F1-score

- False-positive rate
- False-negative rate
- False accepts per hour
- Detection latency

Modern Trigger-Word Models

Modern systems may use:

- CNNs
- CRNNs
- Temporal convolutional networks
- Small Transformer models
- On-device neural networks

Practical Project

A useful exercise is to build a custom trigger-word detector for a phrase such as:

“Computer activate”

Possible pipeline:

Audio recordings

→ Log-Mel spectrogram

→ GRU / CNN

→ Frame-level trigger probability

→ Threshold

→ Action